Names: Jerrod Meltzer, Cole Myers, Rahul Siddhardha Putta
April 20th, 2026

Term Project Final Report:
Haskell BlackJack Game

Section #1: Introduction

The goal of this project was to design and implement a fully functional Blackjack game using the Haskell programming language. Blackjack is a widely recognized card game that combines elements of probability, strategic decision-making, and real-time interaction, making it an excellent candidate for exploring both theoretical and practical aspects of programming languages. The game requires handling randomness through card shuffling, managing game state across multiple turns, processing user input, and applying rule-based logic to determine outcomes. These characteristics make Blackjack a meaningful and sufficiently complex problem for evaluating the strengths and challenges of a programming language.

This project was motivated by two primary objectives. The first objective was to gain hands-on experience with Haskell, a purely functional programming language that differs significantly from more commonly used imperative and object-oriented languages such as Java and Python. Unlike these languages, Haskell emphasizes immutability, function composition, and declarative problem-solving, which requires a fundamentally different way of thinking about program design. By working through a complete application, this project provided an opportunity to better understand Haskell's syntax, type system, and execution model, while also reinforcing core functional programming concepts such as recursion, higher-order functions, and lazy evaluation.

The second objective was to explore how functional programming paradigms handle real-world programming challenges, particularly those involving randomness, user interaction, and sequential decision-making. In traditional imperative languages, these problems are typically addressed using loops, mutable variables, and step-by-step state updates. In contrast, Haskell approaches these challenges through recursion, immutable data structures, and controlled side effects using IO operations. Implementing a Blackjack game required bridging the gap between pure functional logic and real-world input/output behavior, offering valuable insight into how functional languages manage side effects while maintaining referential transparency.

In addition to these objectives, the project also serves as an exploration of software design principles within a functional context. The program was structured into modular components responsible for specific tasks such as card evaluation, deck manipulation, player interaction, and dealer logic. This modular design improves readability, maintainability, and scalability, while also demonstrating how abstraction can be effectively achieved in Haskell through function decomposition. Furthermore, the inclusion of features such as input validation, betting mechanics, and recursive gameplay highlights the ability of Haskell to support increasingly complex application logic.

By implementing Blackjack, this project demonstrates how Haskell manages recursion, immutable data, and IO operations while maintaining code clarity and correctness. It also highlights the trade-offs involved in using a functional language for interactive applications, including both the benefits of strong type safety and the challenges associated with handling state and side effects. Ultimately, this project provides a comprehensive case study of applying functional programming techniques to a real-world problem, reinforcing both theoretical knowledge and practical programming skills.

Section #2: Language Specifications

2.1 Paradigm of the Language

Haskell is a purely functional programming language, meaning that it is built around the concept of functions as the primary building blocks of computation. In Haskell, functions are first-class citizens, allowing them to be passed as arguments, returned from other functions, and stored in data structures. The language avoids mutable state, ensuring that once a value is defined, it cannot be changed. Instead of executing a sequence of commands like in imperative programming, computation in Haskell is based on evaluating expressions. In addition to these core characteristics, Haskell supports several advanced features, including lazy evaluation, which delays computation until results are needed, strong static typing, which helps catch errors at compile time, and referential transparency, which guarantees that expressions can be replaced with their corresponding values without changing the program's behavior.

2.2 Historical Background

Haskell was developed in the late 1980s by a committee of researchers with the goal of creating a standardized functional programming language. It was named after the mathematician Haskell Curry, who is well known for his contributions to combinatory logic and the foundations of functional programming. The language evolved from earlier functional languages such as Miranda, ML, and Lisp, incorporating ideas from each to form a more consistent and expressive system. Over time, Haskell has become widely used in academic and research settings, particularly in areas such as compiler design, type systems, and formal verification, where its strong theoretical foundation and emphasis on correctness make it especially valuable.

2.3 Elements of the Language

Haskell includes a set of reserved words that define the structure and flow of programs. Common examples include keywords such as if, then, and else for conditional expressions; case and of for pattern matching; let and in for local bindings; where for defining helper expressions; and do for sequencing operations within the IO context. These reserved words play a critical role in organizing program logic and enabling Haskell's declarative style.

In terms of data representation, Haskell provides several primitive data types, including Int for integers, Float for floating-point numbers, Bool for boolean values, and Char for individual characters. These basic types serve as the foundation for more complex data structures. Haskell also supports structured types such as lists and tuples, which are widely used in this project. Lists, such as [String], allow for the storage and manipulation of collections of values, making them ideal for representing decks of cards and player hands. Tuples, such as (a, b), group multiple values of potentially different types into a single structure. Additionally, Haskell supports the creation of custom data types, which enable developers to model more complex systems, although they were not heavily utilized in this particular project.

2.4 Syntax Overview

Haskell syntax is structured and highly expressive, relying heavily on indentation to define the layout and scope of code rather than using braces or semicolons. This indentation-based structure enforces readability and consistency across programs. The language makes extensive use of pattern matching, which allows functions to be defined by directly specifying the structure of their inputs, leading to clear and concise logic. Additionally, Haskell uses type signatures to explicitly define the types of functions, improving code clarity and enabling strong compile-time type checking. For example, a function signature such as handValue :: [String] -> Int indicates that the function takes a list of strings and returns an integer. Functions in Haskell are typically defined using equations, such as cardValue card = ..., where the behavior of the function is described in terms of input-output relationships rather than step-by-step instructions.

2.5 Control Abstractions

Haskell syntax is structured and highly expressive, relying heavily on indentation to define the layout and scope of code rather than using braces or semicolons. This indentation-based structure enforces readability and consistency across programs. The language makes extensive use of pattern matching, which allows functions to be defined by directly specifying the structure of their inputs, leading to clear and concise logic. Additionally, Haskell uses type signatures to explicitly define the types of functions, improving code clarity and enabling strong compile-time type checking. For example, a function signature such as handValue :: [String] -> Int indicates that the function takes a list of strings and returns an integer. Functions in Haskell are typically defined using equations, such as cardValue card = ..., where the behavior of the function is described in terms of input-output relationships rather than step-by-step instructions.

2.6 Abstractions Mechanisms

Haskell supports abstraction through several key mechanisms that promote modularity and code reuse. At its core, abstraction is achieved through functions, which allow complex logic to be broken down into smaller, manageable components. The language also supports higher-order functions, such as map and filter, which take other functions as arguments and enable powerful operations over data collections. Additionally, Haskell provides a module system that allows programs to be organized into separate components, although this feature was not heavily utilized in this particular project.

In the context of this Blackjack implementation, abstraction is primarily achieved through the use of functions that encapsulate specific aspects of the game logic. Functions such as play, dealerPlay, and playerTurn each handle distinct responsibilities, making the overall program easier to understand and maintain. Furthermore, the use of mapM_demonstrates how iteration over lists can be handled in a functional style, allowing actions to be applied to each element without relying on traditional looping constructs.

2.7 Writability, Readability, Reliability

Haskell can be evaluated in terms of writability, readability, and reliability, each of which highlights different strengths and challenges of the language. In terms of writability, Haskell presents a moderate level of difficulty. Its strict syntax rules and reliance on indentation require careful attention to formatting, and the functional programming style often demands a shift in mindset for those accustomed to imperative or object-oriented languages. Developers must think in terms of expressions, recursion, and immutability rather than step-by-step execution, which can initially slow down development.

In contrast, Haskell offers high readability once the syntax and paradigms are understood. Programs are often composed of clear and concise function definitions, and the strong type system makes it easier to understand what each function is intended to do. Type signatures serve as a form of documentation, improving clarity for both the original developer and others reading the code. However, readability can decrease in more complex sections of code, particularly when multiple expressions are nested or when advanced functional patterns are used.

Haskell excels in reliability due to several key features. Its strong static type system catches many errors at compile time, reducing the likelihood of runtime failures. Additionally, the absence of mutable state ensures that data cannot be altered unexpectedly, which helps prevent bugs related to state changes. Combined with referential transparency, these characteristics make Haskell programs highly predictable and dependable, contributing to a robust and error-resistant development environment.

2.8 Strengths and Weaknesses

Haskell exhibits several notable strengths that make it a powerful language for building reliable and well-structured programs. One of its primary advantages is its strong type safety, which allows many errors to be detected at compile time rather than during execution. This contributes to more robust and predictable software. Additionally, Haskell enables concise code, as many complex operations can be expressed in fewer lines through the use of higher-order functions and functional composition. Another key strength is the ease of reasoning about program behavior, largely due to the absence of side effects in pure functions. Because functions do not modify external state, developers can more easily understand and verify how different parts of the program will behave.

Despite these strengths, Haskell also presents several challenges. The language can be difficult for beginners, particularly those who are unfamiliar with functional programming concepts such as recursion, immutability, and higher-order functions. This learning curve can make initial development slower compared to more traditional languages. Additionally, handling input and

output is more complex in Haskell due to the use of the IO monad, which requires a different approach to managing side effects. Debugging can also be more challenging, as errors are often related to type mismatches or abstract functional constructs, which may not be immediately intuitive. Overall, while Haskell offers significant benefits in terms of correctness and expressiveness, it requires a deeper understanding of its underlying principles to use effectively.

2.9 Program Overview

The Blackjack program developed in this project incorporates several core components that work together to simulate a complete and interactive game experience. The program begins with deck creation, where a full set of playing cards is represented using a list structure. This deck is then randomized through a shuffling process that introduces unpredictability into the game. Once shuffled, the program handles the dealing of cards by distributing initial hands to both the player and the dealer while maintaining the remaining deck for future use. The player is then given control through a decision-making system that allows them to choose between actions such as hit, stand, or double down. These decisions directly impact the progression of the game and the player's final outcome.

In addition to player interaction, the program includes automated dealer behavior that follows standard Blackjack rules. The dealer draws cards until reaching a hand value of at least 17, ensuring consistent and rule-based gameplay. The program also includes a hand value calculation system that accounts for special cases, such as Aces, which can be valued as either 1 or 11 depending on the context of the hand. This dynamic adjustment is essential for maintaining accurate game logic and fairness.

The implementation of this program highlights several important features of the Haskell programming language. Recursion is used extensively, particularly in functions such as the dealer's turn, to replace traditional looping constructs. Input and output operations are handled using functions like getLine and putStrLn, demonstrating how Haskell manages interaction with the external environment. The program also makes significant use of list manipulation for managing the deck and player hands, as well as pattern matching to safely handle different data

structures and conditions. Additionally, error handling is implemented through the use of readMaybe, which ensures that invalid user input is managed gracefully without causing the program to crash. Together, these features showcase the expressive power and functional design principles of Haskell in building a real-world application.

```haskell
playerTurn :: [String] -> [String] -> Int -> IO ([String], [String], Int)
playerTurn players_cards play_deck currentBet = do
    showHand "You have:" players_cards
    putStrLn $ "Current bet: $" ++ show currentBet

    if handValue players_cards > 21 then
        return (players_cards, play_deck, currentBet)
    else do
        putStrLn "What do you want to do?"
        putStrLn "1. stand"
        putStrLn "2. hit"
        putStrLn "3. double down"
        decisionStr <- getLine

        case readMaybe decisionStr :: Maybe Int of
            Nothing -> do
                putStrLn "Wrong input."
                playerTurn players_cards play_deck currentBet

            Just decision ->
                if decision == 1 then
                    return (players_cards, play_deck, currentBet)
                else if decision == 2 then do
                    let (new_players_cards, new_play_deck) = play decision players_cards play_deck
                    playerTurn new_players_cards new_play_deck currentBet
                else if decision == 3 then do
                    let (new_players_cards, new_play_deck) = play decision players_cards play_deck
                    let doubledBet = currentBet * 2
                    putStrLn $ "Double down! Bet is now $" ++ show doubledBet
                    return (new_players_cards, new_play_deck, doubledBet)
                else do
                    putStrLn "Wrong input."
                    playerTurn players_cards play_deck currentBet
```

(Example of Recursive Element Within Our Program – Player Turn Implementation)

Section #3: Problem Definition (Use Cases)

The problem addressed in this project is the simulation of a Blackjack game that accurately models the interactions and rules of real gameplay. In the first use case, the player participates directly in the game by receiving two initial cards, making decisions such as hitting, standing, or doubling down, and ultimately having the game determine the outcome based on their final hand. In the second use case, the dealer operates automatically according to standard Blackjack rules, drawing additional cards until reaching a hand value of at least 17. In the third use case, a betting system is incorporated to enhance realism, where the player begins with a base bet and has the option to double their bet when choosing to double down. Together, these use cases define the core functionality of the system and ensure that both player interaction and rule-based game mechanics are properly simulated.

Section #4: Proposed Method

4.1 Intuition

The Blackjack program is designed using core principles of functional programming. The deck and player hands are represented using lists, which provide a simple and effective structure for storing and manipulating collections of cards. Game actions such as dealing, drawing cards, and evaluating outcomes are implemented through functions, allowing each part of the program to remain modular, reusable, and easy to reason about. This functional decomposition makes the codebase more organized and simplifies debugging and future extensions.

Instead of using traditional looping constructs, the program relies on recursion to handle repeated actions such as drawing cards or progressing through turns. This approach aligns with the functional paradigm and reinforces a declarative style of programming, where the focus is on what the program should accomplish rather than how to explicitly control each step. Additionally, higher-level operations such as mapping and filtering are used to process collections of data, further emphasizing abstraction and code clarity.

A key advantage of this design is the elimination of mutable state. Because data is not modified in place but instead passed through functions as new values, the program maintains a consistent and reliable flow of information. This immutability ensures that previous states remain unchanged, which helps prevent logical errors and makes the behavior of the program easier to predict. Furthermore, side effects such as user input and output are carefully managed through Haskell's IO system, allowing the program to interact with the user while still preserving the integrity of its functional core.

Overall, this approach demonstrates how functional programming can effectively model a real-world interactive system, balancing pure computation with controlled side effects while maintaining clarity, correctness, and maintainability.

4.2: Algorithm Description

The overall algorithm of the program consists of several interconnected components that work together to simulate the flow and logic of a Blackjack game. Each component is designed to handle a specific part of the gameplay, and together they create a cohesive and functional system. The execution of the program follows a structured sequence, beginning with deck

preparation and continuing through player interaction, dealer behavior, and final outcome evaluation.

The shuffle operation is responsible for introducing randomness into the game. It works by randomly selecting elements from the deck and recursively constructing a new list, ensuring that the order of cards is unpredictable for each round. This step is essential for maintaining fairness and realism in the simulation. Once the deck has been shuffled, the deal operation selects the first four cards and distributes them between the player and the dealer, while the remaining cards are preserved for subsequent draws. This establishes the initial game state and sets up both participants for the round.

During the player's turn, the algorithm processes user input to determine the next action. The player is repeatedly prompted to choose between standing, hitting, or doubling down. Based on the selected action, the program updates the player's hand by drawing additional cards from the deck if necessary. This process is implemented recursively, allowing the program to continue handling decisions until the player either chooses to stand or exceeds the maximum hand value of 21. Input validation is also incorporated to ensure that only valid decisions are accepted, improving the robustness of the system.

The dealer's behavior is handled automatically according to standard Blackjack rules. After the player's turn is complete, the dealer reveals their full hand and begins drawing cards if their total value is below 17. This logic is implemented using recursion, which continues until the dealer reaches the required threshold or the deck is exhausted. Finally, the program compares the values of the player's and dealer's hands to determine the outcome of the game, accounting for conditions such as busts, wins, losses, and ties.

Together, these components form a complete and well-structured algorithm that accurately models the decision-making process and rule enforcement of Blackjack. The use of recursion, functional decomposition, and controlled IO operations allows the program to maintain clarity while effectively handling the complexity of the game.

```haskell
dealerPlay :: [String] -> [String] -> ([String], [String])
dealerPlay dealers_cards play_deck
    | handValue dealers_cards < 17 =
        case play_deck of
            [] -> (dealers_cards, play_deck)
            (x:xs) -> dealerPlay (dealers_cards ++ [x]) xs
    | otherwise = (dealers_cards, play_deck)

showHand :: String -> [String] -> IO ()
showHand label cards = do
    putStrLn label
    mapM_ putStrLn cards
    putStrLn $ "Value: " ++ show (handValue cards)

getBet :: IO Int
getBet = do
    putStrLn "Enter your bet amount:"
    input <- getLine
    case readMaybe input :: Maybe Int of
        Nothing -> do
            putStrLn "Invalid input. Enter a number."
            getBet
        Just bet ->
            if bet <= 0 then do
                putStrLn "Bet must be positive."
                getBet
            else return bet
```

(Code Shown Based on the User Hand)

Section #5: Experiments

The Blackjack program was tested under a variety of conditions to evaluate its correctness, robustness, and overall reliability. These tests were designed to ensure that the game logic behaves as expected across typical gameplay scenarios as well as less common edge cases. By systematically evaluating different inputs and game states, the program's ability to handle real-world usage was assessed.

One area of testing focused on different player decisions, including standing, hitting, and doubling down. These tests verified that the program correctly updates the player's hand, adjusts the deck accordingly, and transitions smoothly between game states. Additionally, the recursive structure of the player turn logic was examined to ensure that repeated actions are handled correctly without causing errors or unexpected behavior.

The program was also tested against edge cases, such as situations where the deck becomes empty or nearly depleted. These scenarios are important for validating that the program can gracefully handle limited resources without crashing or producing invalid results. Proper handling of such cases ensures the stability of the game during extended play.

Finally, input validation was thoroughly tested by providing invalid or unexpected user inputs. This included non-numeric values, out-of-range selections, and incorrect formats. The program was designed to detect these inputs and prompt the user again without terminating execution. This improves the user experience and ensures that the program remains resilient in the face of incorrect input.

Overall, these experiments demonstrate that the program performs reliably under a wide range of conditions, maintaining correct functionality while handling both standard gameplay and exceptional scenarios.

Section #6: Test Bed and Evaluation Questions

The testing and evaluation of the Blackjack program were conducted within a controlled development environment using the Haskell programming language. Specifically, the program was executed and tested using the Glasgow Haskell Compiler (GHC) and the interactive environment GHCi. These tools provided a reliable platform for compiling, running, and debugging the program, allowing for efficient identification of errors and validation of functionality. The use of GHCi was particularly helpful for incremental testing of individual functions, while GHC ensured that the complete program compiled and executed correctly as a whole.

To guide the evaluation process, several key questions were established to assess the correctness and effectiveness of the implementation. First, the program was tested to determine whether it correctly calculates hand values, including the proper handling of special cases such as Aces, which can have multiple values depending on the total hand score. Ensuring accurate hand evaluation is critical to maintaining the integrity of the game.

Second, the implementation was evaluated to confirm that recursion effectively replaces traditional looping constructs. Since Haskell does not rely on imperative loops, it was important to verify that recursive functions such as the player turn and dealer logic correctly manage repeated actions without introducing logical errors or infinite recursion.

Finally, the program was assessed to ensure that the dealer follows standard Blackjack rules, particularly the requirement to draw cards until reaching a hand value of at least 17. This evaluation confirmed that the automated behavior of the dealer aligns with expected game mechanics and produces fair and consistent outcomes.

Together, the testing environment and evaluation questions provided a structured framework for validating both the functional correctness and design effectiveness of the Blackjack program.

Section #7: Observations

Through the testing and evaluation process, several important observations were made regarding both the functionality of the program and the characteristics of the Haskell programming language. First, the program successfully simulates the core rules of Blackjack, including card dealing, player decision-making, dealer behavior, and outcome determination. The logic for calculating hand values, handling Aces, and enforcing game rules was found to be accurate and consistent across multiple test scenarios, indicating that the implementation correctly models the intended gameplay.

Another key observation is that recursion effectively replaces traditional looping constructs commonly found in imperative languages. Functions such as the player turn and dealer logic demonstrate how repeated actions can be handled in a clean and structured manner without the need for mutable counters or loop variables. This reinforces the strength of functional programming in expressing iterative processes through recursive definitions.

However, it was also observed that handling input and output in Haskell is more complex compared to languages like Java or Python. The use of the IO monad introduces additional structure that must be carefully managed, particularly when combining pure functions with side effects such as user interaction. While this design improves reliability and maintains functional purity, it can make the code more difficult to write and understand for beginners.

Finally, the implementation of randomness required the use of external libraries, such as the System.Random module. Unlike some other languages where randomness is more directly accessible, Haskell requires explicit handling of random number generation within the IO context. This adds an extra layer of complexity but also provides greater control over how randomness is incorporated into the program.

Overall, these observations highlight both the strengths and challenges of using Haskell for building an interactive application, demonstrating its ability to produce correct and reliable programs while also requiring careful management of more advanced concepts.

Section #8: Conclusions

This project successfully demonstrated the design and implementation of a functional Blackjack game using the Haskell programming language. Through this process, key concepts of functional programming were applied in a practical context, including recursion, immutability, and the use of higher-order functions. The program effectively simulates core Blackjack mechanics such as card shuffling, player interaction, dealer logic, and outcome determination, confirming that Haskell is capable of supporting real-world, interactive applications.

One of the most important takeaways from this project is the contrast between functional programming and more traditional imperative or object-oriented approaches. Instead of relying on loops and mutable state, the program uses recursion and immutable data structures to manage game flow and state transitions. This not only reinforces theoretical concepts but also highlights the benefits of predictability and reliability that come with functional design. At the same time, the project revealed challenges, particularly in handling input/output operations and randomness, which require a deeper understanding of Haskell's IO model.

Additionally, the project emphasized the importance of modular design and abstraction. By organizing the program into well-defined functions responsible for specific tasks, the overall system became easier to read, maintain, and extend. This modular approach also made it possible to isolate and test individual components, improving the overall quality of the implementation.

In conclusion, this project provided valuable insight into both the strengths and limitations of Haskell as a programming language. It demonstrated that while Haskell may present a steeper learning curve, especially for beginners, it offers powerful tools for building correct, efficient, and maintainable programs. Ultimately, the Blackjack game serves as a practical example of how functional programming principles can be applied to solve real-world problems, reinforcing both conceptual understanding and hands-on programming skills.

Section #8: Citations

Haskell.org. *Haskell Language*. Haskell.org, n.d., https://www.haskell.org/.

HaskellWiki. *HaskellWiki*. HaskellWiki, n.d., https://wiki.haskell.org/.

Miran Lipovača. *Learn You a Haskell for Great Good!*. No Starch Press, 2011, https://learnyouahaskell.github.io/.

“System.Random.” *Hackage: The Haskell Package Repository*, n.d., https://hackage.haskell.org/package/random.

Hutton, Graham. *Programming in Haskell*. 2nd ed., Cambridge University Press, 2016.

Peyton Jones, Simon, editor. *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, 2003.